Fast Differentiable Sorting and Ranking: Divide & Conquer PAV

Nirmal Narendra Patel (nnp528)

March 22, 2026

Abstract

This work parallelizes Teddy Koker's implementation of torchsort [1] for the sequence length dimension. For that, we used a Divide-&-Conquer approach to parallelize the Pool Adjacent Violators (PAV) Algorithm [2]. We demonstrated the parallelization and its correctness by conducting wall-clock time benchmarking, correctness unit tests, and classification training on cifar10 [3] using top-K loss. Our implementation is available at https://github.com/nirmal129/Parallel-torchsort.

1 Introduction

Sorting and ranking algorithms are ubiquitous with applications in computer science theory, social networks, ML algorithms, job executors, and many more. With the rise of automatic differentiation [4], many differentiable variants of discrete functions were proposed and are used in different ML workflows. One such discrete function is sorting.

Standard sorting algorithms are piece-wise linear functions with kinks, rendering them non-differentiable. Ranking algorithms on the other hand are piecewise constant functions so their gradients are either 0 or undefined. To address this, Blondel et al. [5] formulated sorting and ranking as linear programs over the permutahedron, the convex hull of all permutations. By introducing strongly convex regularization, they transform sorting and ranking into smooth projection operators that converge to their hard counterparts as the regularization parameter $\epsilon \to 0$. Asymptotically, they have $O(n \log(n))$ time complexity and $O(n)$ space complexity and enjoys exact computation and differentiation through isotonic optimization.

Blondel et al. [5] algorithm was implemented in NumPy. Teddy Koker released a torch package called *torchsort* that implements Blondel et al. algorithms in CUDA [1]. *torchsort* is currently only batch-wise parallel while still performing sequential forward and backward passes of isotonic regression for each sequence.

In this work, we sequence-wise parallelize forward and backward passes of isotonic regression by implementing Divide-&-Conquer PAV for forward pass and full sequence-wise parallel backward pass of PAV.

2 Soft Sort

For brevity, we will only focus on soft-sort with L_2 regularization to present the parallelization. While parallelization is also applied to soft-rank and can extend to KL regularization, we direct the readers to Blondel et al. paper [5] and *torchsort* [1] for the details of the algorithm. Standard "sort" is applied in descending order and "gather" is a standard operation. The forward and backward algorithms are given in algorithm 1 and algorithm 2, respectively.

Algorithm 1 Soft Sort Forward Pass

Input: $\mathbf{x} \in \mathbb{R}^n, \epsilon$
Output: $\mathbf{y} \in \mathbb{R}^n$ sorted and differentiable
$\mathbf{w} \in \mathbb{R}^n : w_i = (n - i + 1)/\epsilon$
$\mathbf{s}, \boldsymbol{\pi} \in \mathbb{R}^n : \mathbf{s}, \boldsymbol{\pi} \leftarrow \text{sort}(-\mathbf{x})$
$\boldsymbol{\theta} \in \mathbb{R}^n : \boldsymbol{\theta} \leftarrow \text{IsotonicRegression}(\mathbf{w} - \mathbf{s})$
$\mathbf{y} \leftarrow \boldsymbol{\theta} - \mathbf{w}$
Save $\mathbf{s}, \boldsymbol{\theta}, \boldsymbol{\pi}$ for Backward Pass

Algorithm 2 Soft Sort Backward Pass

Input: $\boldsymbol{\nabla}\mathbf{y} \in \mathbb{R}^n$, (Saved) $\mathbf{s}, \boldsymbol{\theta}, \boldsymbol{\pi}$
Output: $\boldsymbol{\nabla}\mathbf{x} \in \mathbb{R}^n$
$\boldsymbol{\nabla}\mathbf{s} \in \mathbb{R}^n : \boldsymbol{\nabla}\mathbf{s} \leftarrow \text{IsotonicRegressionBackward}(\mathbf{s}, \boldsymbol{\theta}, \boldsymbol{\nabla}\mathbf{y})$
$\boldsymbol{\nabla}\mathbf{x} \leftarrow \text{gather}(\boldsymbol{\nabla}\mathbf{s}, \boldsymbol{\pi}^{-1})$

3 Isotonic Regression

For a given $\mathbf{y} \in \mathbb{R}^n$, isotonic regression is optimizing the problem eq. (1).

$$\mathbf{z}_* = \underset{z_1 \geq z_2 \geq \cdots \geq z_n}{\arg\min} \|\mathbf{z} - \mathbf{y}\|^2 \tag{1}$$

The exact solution of eq. (1) can be found using Pool Adjacent Violators Algorithm [2] defined in algorithm 3.

Algorithm 3 Pool Adjacent Violators (PAV)

```
Input: y ∈ R^n
Output: z ∈ R^n
Blocks ← [(y_1), (y_2), ..., (y_n)]          ▷ Initialize: Every element is a 'block'
for each Block in Blocks do
    while Current.Value > Previous.Value do          ▷ Violation Check
        New_Block ← AdjacentMerge(Previous, Current)          ▷ Pool
        New_Block.Value ← Sum(elements) / Count(elements)          ▷ Average
        Current ← New_Block          ▷ Backtrack
    end while
end for
Blocks expanded to vector z
```

4 Parallelizing PAV

4.1 PAV Forward Pass

Divide-&-Conquer approach parallelizes PAV. For the base case, if length of input vector $\mathbf{y}$ is 1 then just return that. If the length is greater than 1 then first split the input into two halves and run parallel PAV on each simultaneously as there is no inter-half coupling. Then merge the results of each half. For merge, observe that rather than operating on individual elements as in sequential algorithm, the merge has to operate on PAV blocks, which can normally be heavily outnumbered by elements. Also, as left and right halves are already outputs of PAV, the violation check should start at the junction. The parallel PAV and merge are given in algorithm 4 and algorithm 5, respectively.

Algorithm 4 Parallel Pool Adjacent Violators

Input: $\mathbf{y} \in \mathbb{R}^n$
Output: $\mathbf{z} \in \mathbb{R}^n$
if length($\mathbf{y}$) $= 1$ **then** ▷ Base Case
    $\mathbf{z} \leftarrow \mathbf{y}$
else
    left_half, right_half $\leftarrow \mathbf{y}$ ▷ Divide
    $L_blocks, R_blocks \leftarrow$ Parallel_PAV(left_half), Parallel_PAV(right_half)
    $\mathbf{z} \leftarrow$ Merge(L_blocks, R_blocks) ▷ Conquer
end if

Algorithm 5 Merge Algorithm for PAV

Input: L_blocks, R_blocks
Output: $Merged_blocks$
while $L_blocks.last < R_blocks.first$ **do** ▷ Junction Violation Check
    $B_1, B_2 \leftarrow L_blocks.pop(), R_blocks.pop_first()$ ▷ Pool
    $merged \leftarrow WeightedMean(B_1, B_2)$ ▷ Average
    $L_blocks.push(merged)$ ▷ Backtrack
end while
$Merged_blocks \leftarrow$ concatenate(L_blocks, R_blocks)

4.1.1 PRAM Analysis

Given p processors, the sequence is divided into p chunks. Sequential PAV is applied to each chunk simultaneously and then $\log(p)$ tree reduction takes place with parallel merges at each level. The work and time complexities depend on the worst complexity of merges at each level. In best case (non-decreasing $\mathbf{y}$), base case and each merge produce a single mega block while in worst case (non-increasing $\mathbf{y}$), all elements are preserved in base case and after each merge. We define average case where base case and each merge preserves half of the elements, i.e. #blocks = #elements/2. All cases have base case work complexity of $O(n)$ and time complexity of $O(n/p)$. Level is defined using L, where $L = 0$ indicates base case and $L = 2^{\log(p)} = p$ indicates final merge level. The PRAM complexities of the three cases is given in table 1.

	Best	**Worst**	**Average**
Single Merge	$O(1)$	$O(2^L \frac{n}{p})$	$O(\frac{n}{p})$
Work	$O(n + 2p - 1)$	$O(n + n\log(p))$	$O(n(3 - \frac{1}{p}))$
Time	$O(\frac{n}{p} + \log(p))$	$O(n(2 - \frac{1}{p}))$	$O(\frac{n}{p}(1 + \log(p)))$
Speedup	$O(\frac{np}{n+p\log(p)})$	$O(\frac{p}{2p-1})$	$O(\frac{p}{1+\log(p)})$
Efficiency	$O(\frac{n}{n+p\log(p)})$	$O(\frac{1}{2p-1})$	$O(\frac{1}{1+\log(p)})$

Table 1: PRAM Complexities

Conclusively, parallel PAV is not work optimal and the benefit of using it is reciprocal to how close $\mathbf{y}$ is to PAV solution. For best and worst case, the complexities worsen as $p \approx n$. Regardless, for $p \approx n$ in average case, work complexity converges and time complexity becomes logarithmic.

4.2 PAV Backward Pass

The backward pass of PAV is provided in algorithm 6. As the operations consist of identifying start of each blocks (simple neighbor comparison), averaging, and scattering, it is trivial to embarrassingly parallelize it across the sequence dimension.

Algorithm 6 PAV Backward Pass

Input: $\mathbf{z}, \boldsymbol{\nabla}\mathbf{z} \in \mathbb{R}^n$
Output: $\boldsymbol{\nabla}\mathbf{y} \in \mathbb{R}^n$
Blocks $\leftarrow$ IdentifyConstantSegments($\mathbf{z}$) ▷ Start of each PAV block
for Block in Blocks **do**
  $\overline{\nabla\Sigma} \leftarrow \sum_{i=\text{Block.Start}}^{\text{Block.End}} (\nabla z)_i / \text{Block.Size}$ ▷ Average
  $\boldsymbol{\nabla}\mathbf{y}[\text{Block.Start} : \text{Block.End}] \leftarrow \overline{\nabla\Sigma}$ ▷ Scatter
end for

5 Results

To test our parallel PAV implementation, we ran the unit-tests provided in *torchsort* [1]. It failed for two cases; however, the failure is isolated to a single element in the output tensor while all other values match the sequential output. We strongly believe the bug is a misaligned pointer in our CUDA kernel, though we were unable to conclusively identify it.

To verify compatibility in a training pipeline, we ran classification training on CIFAR-10 [3] using top-K loss relying on soft-rank and compared the evaluation curves, shown in fig. 1, of sequential and parallel implementations. Both curves follow the same trend-line and differ only by training noise, confirming behavioral correctness. Since CIFAR-10 operates on sequences of length 10, no wall-clock speedup is expected or observed.

Finally, we benchmarked wall-clock time against the sequential implementation and `torch.sort` [6], fixing batch size to 4 for varying sequence lengths and sequence length to 512 for varying batch size. Results are shown in fig. 2. The left plot demonstrates clear gains from parallelization, particularly from the embarrassingly parallel backward pass. The right plot reveals suboptimal batch scaling, attributable to poor compute core allocation and lack of kernel fusion, both beyond the scope of this work.

6 Conclusion And Future Directions

In conclusion, we parallelized *torchsort* along the sequence dimension on top of the batch parallelism. While our implementation contains some bugs, we demonstrate considerable proof of work by benchmarking the parallel implementation and incorporating it in a ML training pipeline.

Our implementation is still naive and the merge step can be improved further by considering the equivalency between PAV and ROC-Convex-Hull (ROCCH) [7]. PAV is equivalent to 1) $\mathbf{Y} = \text{scan}(\mathbf{y})$, 2) find the upper convex hull of (i, Y_i), 3) scatter the slopes. To find the upper convex hull, implementations like CudaHull [8] or CudaChain [9] can be modified and optimized for PAV.

CudaHull or CudaChain supports general convex hull finding. In case of PAV, the hulls have a particular structure. Suppose, left $\mathbf{L}$ and right $\mathbf{R}$ hulls of PAV algorithm are given then they both will have sorted x-coordinates and the maximum x-coordinate of the $\mathbf{L}$ will be strictly less than the minimum x-coordinate of the $\mathbf{R}$. While running toy experiments, we made an observation that the heat map of slopes between pair of nodes, one from $\mathbf{L}$ and one from $\mathbf{R}$ ($\mathbf{L}$'s nodes are represented by row indices and $\mathbf{R}$'s nodes are represented by column indices), has uni-modal convex down rows, the ridge representing row-maximas ($f(l) = \max_r \text{slope}(L_l, R_r)$) is uni-modal convex up and the common tangent or the connector corresponds to the minima of the ridge. Generalizing it, we conjecture that the common tangent or the connector corresponds to the minima of row maximas, i.e. $(l, r) = \arg\min_l \arg\max_r \text{slope}(L_l, R_r)$ (this maybe already proven in the literature, however,

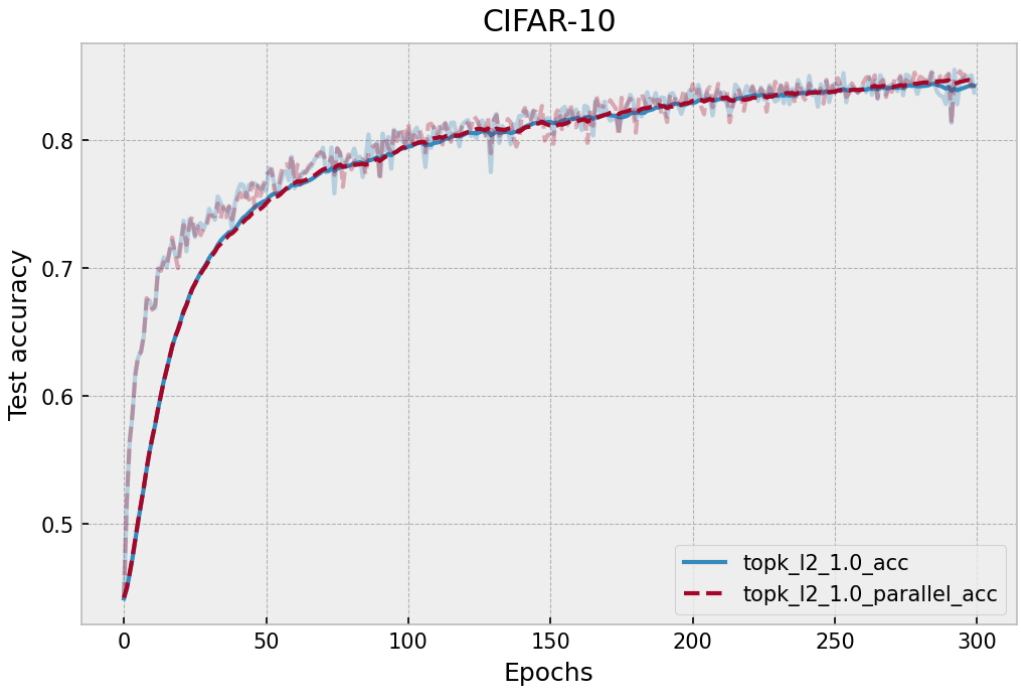

Figure 1: Cifar10 [3] evaluation curves

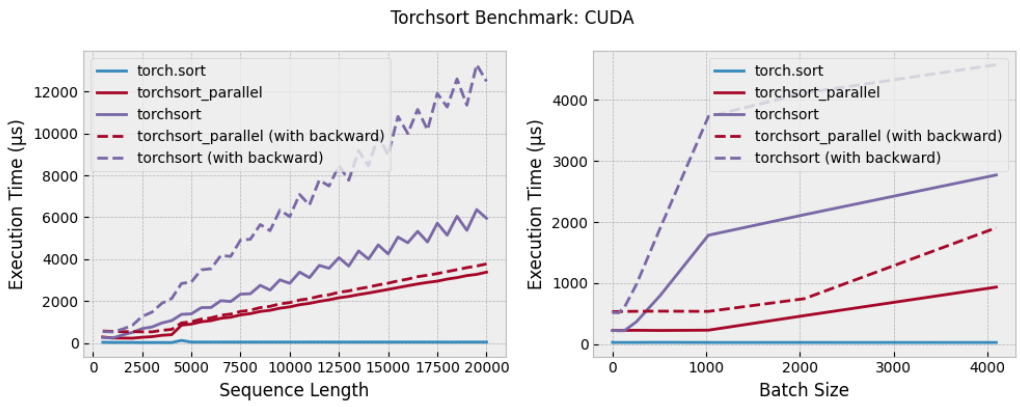

Figure 2: Wall-clock time benchmarking

we didn't find any exact match). Assuming the conjecture is correct, the common connector can be found using nested ternary search in $O(\log(|\mathbf{L}|)\log(|\mathbf{R}|))$ time. Thus, the time complexities of PAV using optimized merge are given in table 2.

Case	**Time**
Best	$O(\frac{n}{p} + \log(p))$
Worst	$O(\frac{n}{p} + \log(2p)\left[\log(n)\log(\frac{n}{p}) + \frac{\log(p^{)}\log(2p^2)}{6}\right])$
Average	$O(\frac{n}{p} + \log(p)(\log(\frac{n}{p}))^2)$

Table 2: Optimized PAV

References

[1] Teddy Koker, Louis Jalouzot, and Robin Betz. teddykoker/torchsort. https://github.com/teddykoker/torchsort, Jun 9 2025.

[2] Jan de Leeuw, Kurt Hornik, and Patrick Mair. Isotone optimization in r: Pool-adjacent-violators algorithm (pava) and active set methods. *Journal of Statistical Software*, 32(5):1–24, 2009.

[3] Alex Krizhevsky. Learning multiple layers of features from tiny images. https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf, 2009.

[4] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. https://arxiv.org/abs/1502.05767, 2018.

[5] Mathieu Blondel, Olivier Teboul, Quentin Berthet, and Josip Djolonga. Fast differentiable sorting and ranking. https://arxiv.org/abs/2002.08871, 2020.

[6] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019.

[7] Tom Fawcett and Alexandru Niculescu-Mizil. Pav and the roc convex hull. *Machine Learning*, 68(1):97–106, Jul 2007.

[8] Ayal Stein, Eran Geva, and Jihad El-Sana. Cudahull: Fast parallel 3d convex hull on the gpu. *Computers & Graphics*, 36(4):265–271, 2012. Applications of Geometry Processing.

[9] Gang Mei. Cudachain: an alternative algorithm for finding 2d convex hulls on the gpu. *SpringerPlus*, 5(1), May 2016.